

Mobile-Optimized 3D Object Scanner

A Step-by-Step Godot Guide for High-Performance Sci-Fi Scan Lines

Visualizing the Effect in the Engine

When this effect is triggered, a bright neon-cyan laser band materializes at the top of your 3D object. As the band sweeps smoothly downwards, it illuminates the object's original textures beneath it while projecting a faint sci-fi grid pattern directly inside the beam. The areas above and below the beam remain completely normal. Because this uses mathematical UV coordinates instead of full-screen post-processing, it renders at a flawless 60 FPS on lower-end mobile devices.

Phase 1: The Scanner Shader

This effect relies on a UV-based Spatial Shader that calculates a glowing line based on the mesh's vertical coordinates.

1. Create the Spatial Shader

Select your 3D mesh, assign a new ShaderMaterial, create a new Shader, and paste this mobile-optimized code:shader_type spatial;

```
uniform sampler2D albedo_texture : source_color;
uniform vec4 scan_color : source_color = vec4(0.0, 1.0, 0.8, 1.0); // Sci-Fi Cyan
uniform float scan_position : hint_range(-0.1, 1.1) = 0.0;
uniform float scan_width : hint_range(0.0, 0.2) = 0.05;

void fragment() {
    // Get the object's base texture
    vec4 base_color = texture(albedo_texture, UV);

    // Calculate the distance from the current pixel to the scan line
    float dist = abs(UV.y - scan_position);

    // Create a soft glowing line using smoothstep
    float scan_line = smoothstep(scan_width, 0.0, dist);

    // Add a faint high-frequency grid pattern inside the laser for extra sci-fi detail
    float grid = clamp(sin(UV.x * 100.0) * sin(UV.y * 100.0), 0.0, 1.0);
    scan_line *= (grid + 0.2);

    // Apply the base color, mixed with the scanning laser
    ALBEDO = mix(base_color.rgb, scan_color.rgb, scan_line);

    // Make the laser physically glow (Requires Additive Glow/Bloom on the camera)
    EMISSION = scan_color.rgb * scan_line * 2.5;
}
```

Phase 2: Material Configuration

Once the shader code is compiled, you must configure its parameters in the editor so it interacts properly with your 3D model.

2. Configure the Inspector Settings

In the Godot Inspector, look under your new Shader Parameters section and apply these settings:

Albedo Texture: Assign your object's standard color texture so the model looks normal when not being scanned.

Scan Color: Change the color to match your drone's UI or laser color (e.g., bright cyan or green).

Scan Width: Adjust this value to make the laser band thicker or thinner.

Phase 3: Animating the Scan

Shader parameters remain static until updated by the engine. You will use a Godot Tween to smoothly animate the scan line from the top of the object to the bottom.

3. Attach the GDScript Logic

Attach this script to your mesh. When you call the `play_scan_effect()` function, it smoothly transitions the `scan_position` uniform over 2 seconds.`extends MeshInstance3D`

```
func play_scan_effect():
    var material = get_surface_override_material(0)
    var tween = create_tween()

    # Reset the scan to the top (0.0)
    material.set_shader_parameter("scan_position", 0.0)

    # Animate to the bottom (1.0) over 2.0 seconds
    tween.tween_method(set_shader_value.bind(material), 0.0, 1.0, 2.0)

func set_shader_value(value: float, mat: ShaderMaterial):
    mat.set_shader_parameter("scan_position", value)
```